

Project Executable Files

Date	15 March 2026	Team ID	SB_INT_2026_PSAI
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant	Maximum Marks	3 Marks

Project Executable Files: This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA (In-Memory Session Storage)
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes (Local / Render Deployed)
7	Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Overview of PocketSmart AI project's file and folder structure:

```
POCKETSMART-AI/  
├── static/  
│   ├── uploads/      # Uploaded outfit images for Jewelry Planner  
│   └── styles.css     # Frontend styling and custom layouts  
└── templates/
```

```

├── dashboard.html    # User Dashboard view
├── history.html      # Past recommendation logs and history
├── home_planner.html # Home Interior Planner interface
├── index.html        # Main landing page
├── jewelry_planner.html # Jewelry Planner interface with image upload
├── login.html        # Login page template
├── party_planner.html # Party Planner interface
├── register.html     # User registration template
├── .env              # Environment variables (API Keys, JWT Secrets)
├── app.py            # Main FastAPI application routes & execution entry
├── gemini_utils.py   # Gemini 1.5 Flash Pro integration & recommendation logic
├── requirements.txt  # Python dependencies (FastAPI, uvicorn, google-generativeai, etc.)
└── README.md        # Project documentation and setup guide

```

Step 3: Deployment / Access Details

Hosted / Deployed URL	http://localhost:8000 / https://pocketsmart-ai.onrender.com
Platform / Provider	Render / FastAPI Server (Uvicorn)
Repository Link	https://github.com/smartbridge/pocketsmart-ai
Login Credentials (Demo)	Username: sai Password: demo123
Hosting Platform	Render / Local Uvicorn Host
Demo Video Link	https://drive.google.com/file/d/pocketsmart-ai-demo/view

Step 4: Run Instructions

Follow these step-by-step instructions to execute PocketSmart AI locally:

Pre-requisites:

- Python 3.9+ installed
- Google Cloud / Google AI Studio API Key with Gemini 1.5 Flash Pro access
- Modern Web Browser (Chrome, Firefox, Edge)

Step 1: Clone the repository

```
git clone https://github.com/smartbridge/pocketsmart-ai.git
cd pocketsmart-ai
```

Step 2: Create and activate virtual environment

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```

Step 3: Install dependencies

```
pip install -r requirements.txt
```

Step 4: Configure environment variables

Create a .env file in the root directory:

```
GOOGLE_API_KEY=your_gemini_api_key_here  
SECRET_KEY=your_jwt_secret_key_here
```

Step 5: Start the application

```
python app.py
```

```
# Or run using uvicorn directly:
```

```
# uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

Step 6: Access the application

Open browser at <http://localhost:8000>

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	API Rate limits on free tier Gemini API calls.	Implemented fallback default recommendations when API limits are exceeded.
2	Image processing requires specific formats (JPEG/ PNG).	Added image validation in backend using Pillow (PIL) before sending to Gemini.
3	In-memory session storage resets on server restart.	Sufficient for current scope; can be migrated to SQLite/ Redis for production.